CSCE 355, Spring 2026
Programming Assignment

Due Thursday April 16, 2026 at 11:30pm EDT

For the programming portion of the course (15% of your total grade) you are to implement an interpreter for a certain kind of two-counter automaton (**2-CA**, described below). Your program will

1. read from a text file on the command line a description of a 2-CA A, which is given by a sequence of transitions, one per line, then
2. read a sequence of strings from standard input, one per line, outputting for each string w,
   - a complete computational trace of A on input w,
   - an indication of whether A accepts or rejects w.

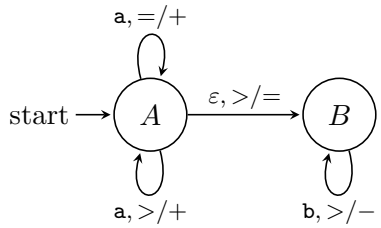
   All output is to standard output. (Standard input is from the keyboard by default but can be redirected from a file by the shell. Standard output is to the terminal screen by default but can be redirected to a file by the shell.)

Download and unzip the archive `prog-proj4.zip`. It includes a working executable solution `2ca` (that runs on the linux lab machines) and a subdirectory `test` with some sample 2-CAs.

1 What is a 2-CA?

A 2-CA, or more generally a k-CA for $k \geq 1$, is analogous to a PDA with empty stack acceptance, but instead of having access to a stack, it has access to k many *counters*, each of which can hold a natural number at any given time (initially zero). You will only implement the kind of k-CA where $k = 2$. The k-CA reads the input string from left to right as usual, and each transition can only depend on the current state, the currently scanned input symbol, and for each counter, *whether or not it is currently zero*. The action performed by a transition includes (possibly) changing state, consuming the scanned input symbol (or not, for ε-transitions) and for each counter, either incrementing it, decrementing it (if nonzero), or leaving it unchanged. The k-CA runs until no transitions are possible. At that point, it accepts if the entire input is consumed and all counters are zero; otherwise, it rejects.

For example, the following 1-CA recognizes $\{\mathtt{a}^n\mathtt{b}^n : n \geq 0\}$:



In each transition, the slash (/) separates the condition (left) from the action (right). So,

- $\mathtt{a}, =/+$ means, "if the next input symbol is a and the counter is *equal to* zero, then consume the a and increment the counter";
- $\mathtt{a}, >/+$ means, "if the next symbol is a and the counter is *greater than* zero, then consume the a and increment the counter";
- $\varepsilon, >/=$ means, "if the counter is positive, then don't consume any input and leave the counter unchanged" (this transition is only allowed if there is no alternative non-ε-transition);
- $\mathtt{b}, >/-$ means, "if the next symbol is b and the counter is positive, then consume the b and decrement the counter".

1.1 Formalities

If the explanation above is reasonably clear to you, you may lightly skim this section at first, referring back to it to resolve any detailed questions you may have.

There are many different and inequivalent ways to define a k-CA. We define a very simple one here, which is *deterministic*. As usual, we let ε denote the empty string, and for alphabet Σ, we define $\Sigma^? := \Sigma \cup \{\varepsilon\}$.

Definition 1. For integer $k \geq 1$, a *k-counter automaton (k-CA)* is a tuple $\langle Q, \Sigma, \delta, q_0 \rangle$ where

- Q is a finite set (the *state set*),
- Σ is an alphabet (the *input alphabet*),
- $q_0 \in Q$ (the *start state*), and
- $\delta : Q \times \Sigma^? \times \{=, >\}^k \to Q \times \{+, =, -\}^k$ is a partial function (the *transition function*).

As an argument to δ, the symbol "=" indicates the corresponding counter is currently zero, and ">" indicates the counter is currently positive (i.e., > 0). In the output of δ, the symbol "+" means increment, "=" means leave unchanged, and "−" means decrement (but don't change the counter if it is already zero).

Here are examples where $k = 2$. For some $q \in Q$ and $a \in \Sigma$, suppose

$$\delta(q, a, =, >) = (r, +, -)$$

for some $r \in Q$. This means:

> "If the current state is q, the head is scanning a, the first counter is zero, and the second counter is positive, then consume the a, go to state r, increment the first counter and decrement the second counter."

If

$$\delta(q, \varepsilon, >, >) = (r, =, =),$$

this means:

> "If the current state is q and both counters are positive, then without consuming an input symbol go to state r, leaving both counters unchanged."

It is possible during a computation that an ε-transition and a non-ε-transition are both available at the same time. To ensure deterministic behavior, we impose the following "greedy" rule:

> An ε-transition should be taken **only** if there is no legal non-ε-transition available.

To define behavior formally, we limit ourselves to 2-CAs. (The definition generalizes to k-CAs in the obvious way.)

Definition 2. Let $A := \langle Q, \Sigma, \delta, q_0 \rangle$ be a 2-CA. An *instantaneous description (ID)* or *configuration* of A is a 4-tuple (q, x, n_1, n_2), where $q \in Q$, $x \in \Sigma^*$, and n_1, n_2 are natural numbers (nonnegative integers).

Given a string $w \in \Sigma^*$, the *initial configuration of A on input w* is $(q_0, w, 0, 0)$ (start state, entire input unread, both counters zero).

A configuration (q, x, n_1, n_2) is *accepting* if and only if $x = \varepsilon$ and $n_1 = n_2 = 0$. Any other configuration is *rejecting*. (The state q can be arbitrary).

Similar to the case of PDAs, the ID (q, x, n_1, n_2) means,

> "The current state is q, the unconsumed portion of the input is x, and the two counter values are n_1 and n_2, respectively."

Next we define one step of a computation as a *partial* successor function on IDs. (A partial function is one that is not necessarily defined on all arguments.) The next definition will make this easier.

Definition 3. For a natural number n, define

$$\mathit{sign}(n) := \begin{cases} > & \text{if } n > 0, \\ = & \text{if } n = 0. \end{cases}$$

Define

$$\Delta(-) := -1, \qquad \Delta(=) := 0, \qquad \Delta(+) := 1.$$

Let A be as in Definition 2. Given a state $q \in Q$, a string $w \in \Sigma^*$, and natural numbers n_1, n_2, the *successor* of the configuration $C := (q, w, n_1, n_2)$ (if it exists) is the configuration defined by the following rules (letting $s_1 := \mathit{sign}(n_1)$ and $s_2 := \mathit{sign}(n_2)$):

- If $w = ax$ for some $a \in \Sigma$ and $x \in \Sigma^*$, then

- if $\delta(q, a, s_1, s_2) = (r, t_1, t_2)$ where $r \in Q$ and $t_1, t_2 \in \{-, =, +\}$, then the successor of C is $(r,\ x,\ \max\{0,\ n_1 + \Delta(t_1)\},\ \max\{0,\ n_2 + \Delta(t_2)\})$;
- if $\delta(q, a, s_1, s_2)$ is undefined but $\delta(q, \varepsilon, s_1, s_2) = (r, t_1, t_2)$, then the successor of C is $(r,\ w,\ \max\{0,\ n_1 + \Delta(t_1)\}, \max\{0,\ n_2 + \Delta(t_2)\})$;
- if both $\delta(q, a, s_1, s_2)$ and $\delta(q, \varepsilon, s_1, s_2)$ are undefined, then C has no successor.

- If $w = \varepsilon$, then
  - if $\delta(q, \varepsilon, s_1, s_2) = (r, t_1, t_2)$ where $r \in Q$ and $t_1, t_2 \in \{-, =, +\}$, then the successor of C is $(r,\ \varepsilon,\ \max\{0,\ n_1 + \Delta(t_1)\}, \max\{0,\ n_2 + \Delta(t_2)\})$;
  - if $\delta(q, \varepsilon, s_1, s_2)$ is undefined, then C has no successor.

For IDs C_1 and C_2, we write, "$C_1 \vdash C_2$" to mean that C_2 is the successor of C_1.

A *computation path* of A on input $w \in \Sigma^*$ is a sequence $C_0 \vdash C_1 \vdash \cdots$ of IDs starting with the initial configuration where each subsequent ID is the successor of the previous one. The path ends when it reaches a configuration with no successor. If the last configuration is accepting, then A *accepts* w; if the last configuration is rejecting, then A *rejects* w. Note that it is possible that the path never ends; in this case, we say that A *loops* on w.

The *language recognized by* A is defined as you would expect:

$$L(A) := \{w \in \Sigma^* : A \text{ accepts } w\}.$$

(By this definition, if A loops on w, then $w \notin L(A)$.)

1.2 Example

Consider the 1-CA given by the transition diagram above.

- On input `aabb`, the computation path is

$$(A, \texttt{aabb}, 0) \vdash (A, \texttt{abb}, 1) \vdash (A, \texttt{bb}, 2) \vdash (B, \texttt{bb}, 2) \vdash (B, \texttt{b}, 1) \vdash (B, \varepsilon, 0) \quad \text{(accept)}$$

- On input `aab`, the computation path is

$$(A, \texttt{aab}, 0) \vdash (A, \texttt{ab}, 1) \vdash (A, \texttt{b}, 2) \vdash (B, \texttt{b}, 2) \vdash (B, \varepsilon, 1) \quad \text{(reject)}$$

- On input `abb`, the computation path is

$$(A, \texttt{abb}, 0) \vdash (A, \texttt{bb}, 1) \vdash (B, \texttt{bb}, 1) \vdash (B, \texttt{b}, 0) \quad \text{(reject)}$$

- On input `bbaa`, the computation path is

$$(A, \texttt{bbaa}, 0) \quad \text{(reject)}$$

- On input ε, the computation path is

$$(A, \varepsilon, 0) \quad \text{(accept)}$$

(By the way, a 1-CA can be easily simulated by a PDA where the size of the stack is used for the counter. Thus 1-CAs can only recognize context-free languages. This is not true for 2-CAs, however; some of the 2-CAs in the test files recognize non-context-free languages.)

2 Input File Format

Your program takes a filename (a text file with the description of a 2-CA) as the only command line argument. The file contains a list of the allowed transitions, one per line. A transition has the following syntax:

```
(<from_state>,<symbol?>,<c1_cond>,<c2_cond>) -> (<to_state>,<c1_action>,<c2_action>)
```

The arguments to the transition function are to the left of the arrow, and the action to be performed is to the right. Items surrounded by angle brackets are not meant literally but instead are meant to be substituted for various syntactic elements:

- `<from_state>` and `<to_state>` are state labels—the state transitioning from and to, respectively. Each consists of a pound sign ("`#`") followed by any string of zero or more alphanumeric characters. The underscore (`_`) counts as an alphanumeric character. *The pound sign by itself denotes the start state.*
- `<symbol?>` is either a single character (the next character to be consumed, which could be any printable character except a space, tab, or newline), or nothing (indicating an ε-transition).
- `<c1_cond>` and `<c2_cond>` are the conditions on counters 1 and 2, respectively. Each is either "`=`" (meaning equal to zero), "`>`" (meaning greater than zero), or "`*`" (a wildcard meaning any value; see below).
- `<c1_action>` and `<c2_action>` are the actions to be performed on counters 1 and 2, respectively. Each is either "`+`" (increment), "`=`" (leave unchanged), or "`-`" (decrement).

All spaces and tabs in a line containing a transition should be ignored. Any occurrence of `//` indicates the start of a comment (running to the end of the line) and should be ignored. Empty lines must be commented.

There will always be at least one transition out of the start state (`#`), but transitions can appear in any order within the file.

2.1 Example

The 1-CA given in Section 1 above is given in the file `ab.txt` in the `test` subdirectory (with different state labels and two counters):

```
// 2-CA to recognize the language { a^nb^n : n >= 0 }
//
// The 2nd counter is unused and stays 0 the whole time.
//
(#,       a,*,=) -> (#,+,=)         // Read an "a" and increment counter 1
(#,        ,>,=) -> (#read_bs,=,=)  // epsilon-transition from # to #read_bs
(#read_bs,b,>,=) -> (#read_bs,-,=)  // Read a "b" and decrement counter 1
```

The second transition is an ε-transition, as there is no non-whitespace character in the 2nd argument. The star (`*`) used in the first transition is a wildcard. That line is shorthand for two separate transitions:

```
(#,a,=,=) -> (#,+,=)
(#,a,>,=) -> (#,+,=)
```

3 Simulation

After you read in a 2-CA from the file and internalize it, your program then reads a sequence of input strings from standard input, one per line, ending with an end-of-file marker (ctrl-D if typing from the linux keyboard). For each input string, you simulate the 2-CA on that string, writing to standard output the computation path, one ID per line, then saying either "ACCEPT" or "REJECT" at the end as appropriate.

Each input string will be at most 128 characters in length.

4 How to Test Your Code

The zip file package includes an executable solution `2ca` that runs on the linux lab machines.[1] It has a verbose option that makes the interaction more user-friendly.

In a linux terminal shell in the directory into which you unzipped the file, type

```
./2ca --verbose test/ab.txt
```

You should first see an internalized description of the 2-CA, followed by a prompt to enter a string. Enter any string of `a`'s and `b`'s and you'll see a complete computation path on that input string followed by whether the string was accepted or rejected. (To enter ε as the input, just hit RETURN.) You can use "`-v`" instead of "`--verbose`" to turn this option on.

Without the verbose option, you'll see no description of the 2-CA, nor any explicit prompt for a string, nor the column headers on the computation path. **Your program should match the behavior of `2ca` *without* the verbose option.** You don't have to line things up neatly in columns, because we will ignore spaces and tabs in your output.

You should test your program in place of `2ca` on this and the other `*.txt` files in the `test` directory and compare the outputs with those given by `2ca`.

5 How Your Program is Graded

The grading of your work will be automated via a script running on linux. Therefore we are requiring you to stick to a simple, uniform interface for your program: Your program must be able to be run via a simple command-line invocation on one of the linux boxes in the department's linux lab, and all I/O will be ASCII text. You may write your program in any programming language you want, provided it is implemented on the linux machines in the CSCE linux lab. To repeat: your program (after compiling, if necessary) should be a stand-alone executable that can be run directly from the linux `bash` shell, requiring no special user interface to execute (e.g., Eclipse).[2] More about this below.

5.1 What the script does

Let's say you have a stand-alone executable called `my_interpreter`. Then the script effectively does the following for the test file `ab.txt`:

[1] The solution does two things that you are *not* required to do: it checks for syntax and semantic errors in the 2-CA file (all test files we give you are error-free).

[2] It's OK if we need to invoke the JVM by preceding your main class name with "java" on the command line, or the python interpreter by preceding your program with "python3".

```
$ ./my_interpreter test/ab.txt < test/ab.in | ./sw.pl > tmp1.txt
$ ./2ca test/ab.txt < test/ab.in | ./sw.pl > tmp2.txt
$ diff tmp1.txt tmp2.txt
```

If `tmp1.txt` and `tmp2.txt` are identical, then `diff` quits with no output, which indicates that your program ran correctly. Otherwise, `diff` returns a list of discrepancies between the two files, line by line. The program `sw.pl` is a Perl script that strips whitespace (spaces and tabs) from each line of standard input and sends the results to standard output. It also strips carriage returns (`\r`) which are included with files produced on DOS/Windows. Our grading script will repeat the above for each 2-CA in the test directory, collecting the outputs of the `diff` commands into a file `comments.txt` that also includes scoring information.

You can run the above commands yourself if you want more clarity on how we will grade you. If the symbols "`<`", "`|`", and "`>`" above are unfamiliar to you, you should read up on input/output redirection and pipes in linux.

You may assume that when we test your program for grading, the 2-CA descriptions will adhere to their respective formats, i.e., you won't need to error-check the files. You can send anything you want to standard error; the grading program will ignore it.[3]

5.2 Style

Your code should be economically written, well-structured, and well-commented, following the common stylistic guidelines of the programming language you use. The code should also be reasonably efficient, but this is a secondary requirement. The script allows a generous 11 seconds for each run of your code. You'll be fine as long as your runs keep to this time limit; otherwise, you won't get credit for that run.

5.3 Summary

As we mentioned, your project will be graded automatically. To grade your project, we will use a script and the test files we have provided to test and grade your project. Just to be perfectly clear: we will grade your project by running a script on it with owner privileges using one of the linux lab machines. We will not run your code personally. The comments produced by that script will determine your grade. This means that you will not get credit for attempting to do something. You will only get credit for what actually works, as determined by the script run on a CSE linux Lab machine.

We will test your code on seven different 2-CAs. Each 2-CA corresponds to two files in the `test` directory: the `.txt` file contains the 2-CA's description, and the `.in` file contains the input strings for the simulation. You earn 15 points for every 2-CA your program handles correctly, except for `match_delim2.txt`, which is worth 20 points. Any incorrect behavior (including time-out) earns zero points for that 2-CA. Your score will not exceed 100 points.

[3] We call the three I/O streams open by default on GNU/Linux programs *standard input* (keyboard input by default), *standard output* (screen output by default), and *standard error* (unbuffered output sent to the screen by default, even if standard input and output are redirected by the system). Some programming environments may use different names for these streams, e.g., C programs using `stdio.h` for high-level I/O call these `stdin`, `stdout`, and `stderr`, respectively; C++ programs typically use `cin`, `cout`, and `cerr` for the same purpose.

6 How to Submit

Submission will be via Blackboard. Upload a single file, either a .zip file or a .tar.gz file, containing

1. all your source code files, which should all be in the same directory, i.e., no subdirectories (and no automatically generated files, please),
2. an optional file `readme.txt` with anything you want to tell us (we will read this with our own eyes at our discretion), and
3. a file `build-run.txt` giving linux (`bash`) shell commands to compile and/or run your program. Don't include the command line arguments (e.g., `test/ab.txt`); we will supply those separately when we run your program. This file should be named `build-run.txt` and placed in the same directory as your other source files. See below for the contents of this file.

IMPORTANT NOTE: You *must* use either the ZIP format (file extension .zip) or the GZIPPED TAR format (file extension .tar.gz) for your submission file. Your file will be de-archived either with `unzip` or with `gunzip; tar -xf`, depending on your file name's extension. Do not use any other archive format, particularly the RAR format, which is proprietary to Windows (I personally do not have Windows on any machine I use). If you deviate from the allowed formats, you risk getting zero credit for the entire assignment. Keep in mind that linux file names are case-sensitive.

6.1 Examples of build-run files

Suppose you implement your program in Java, and your main class is called `MyInterpreter`. Then your `build-run.txt` file would look like this:

```
# Lines like these are comments and will be ignored
Build:
  javac MyInterpreter.java
Run:
  java MyInterpreter
# Don't include command line arguments to the run command!
# The indenting is optional.
```

For another example, suppose you implement your program in C as a single compilation unit called `my_interpreter.c`. Then your `build-run.txt` file would look something like this:

```
Build:
  gcc my_interpreter.c
  mv a.out my_interpreter
Run:
  ./my_interpreter
# Again, no command line arguments, please.  They will be supplied automatically.
```

Note that you can have any number of build commands, and they will be executed in order (in the directory containing your source files) before the run command. Always give the Build commands first before the Run command.

Suppose instead that you have several compilation units for your programs, including shared code, and a complicated build procedure, but you have a single `Makefile` controlling it all, capable of producing an executable called `my_interpreter`. Then the `build-run.txt` file can just look something like this:

```
Build:
  make -B
Run:
  ./my_interpreter
```

(Use the `-B` option or `--always-make` option with `make`; it will build your entire program from source regardless of any intermediate files.)

As a final example, suppose you implement your program in Python, which is a scripting language that can be run directly without a compilation step. Then your `build-run.txt` file might look like this:

```
Build:
Run:
  python3 my_interpreter.py
# You still need to say "Build:" even though there are no build commands.
# For the linux machines you should use python3 instead of python.
```

7 Two Common Pitfalls

No allowances will be made for falling for either of these:

Not testing your code on a lab machine. You tested your code on your laptop and it ran perfectly, so you just submitted without bothering to test it on one of the linux lab machines. You're then shocked to see that you got a zero for the entire assignment. This happens more often than you'd think. Even if your laptop is running linux, it may be configured differently from the machines in the lab. In the labs, you need to use `python3` to invoke Python.

Windows Versus Linux. In text files created in DOS/Windows, a newline is given by two-character sequence: carriage-return/linefeed (in C-like languages, the code is `\r\n`. GNU/Linux systems only use the linefeed character to indicate a new line. The test files are all GNU/Linux-created. Make sure your parser is not expecting carriage-returns (`\r`) in the test files.

The moral is not to wait until the last hour, or even the last day, to test your code on a linux lab machine.

8 Do Your Own Work

The code you write and submit must not come from another person or persons, even in part. You may discuss the homework with others at the conceptual level (see the next paragraph), but you may not copy code either directly or indirectly from any other human source, even if you modify it afterwards. Likewise, you must take all reasonable precautions not to let your code be copied by anyone else, either in this class or in future classes. This includes uploading or developing your

code on a web platform—such as SourceForge or GitHub—in a way that can be seen by others. Violating this policy constitutes a violation of the Carolina Honor Code, and will have serious consequences, including, but not limited to, failure of the course.

Discussing the project with others in the class is allowed (even encouraged), but you must include in your `readme.txt` file the names of those with whom you discussed the project.

8.1 Use of AI

You *are* allowed make use of AI for this project without penalty, `but` if you do, you *must* include in your `readme.txt` file what resource(s) you used and the specific queries you made to that resource. Failing to do so constitutes violation of class policy and the Carolina Honor Code, with the same consequences as given above.

Please note that this means you must retain the text of your queries for inclusion in the `readme.txt` file.

8.2 Ask

If you have any questions about what this policy means, please review the relevant section of the course syllabus or ask me.